

JAVA OOP PRACTICAL RECORD

Bank Account Management System

Experiment No.	04
Date	17/09/2026
Title	Bank Account Management System using Inheritance, Method Overriding and Switch-Case

1. Objective / Aim

To write a Java program that implements a Bank Account Management System using object-oriented programming concepts such as parameterized constructors, inheritance, method overriding, object creation, fund transfer between accounts, and a menu-driven switch-case structure.

2. System Requirements

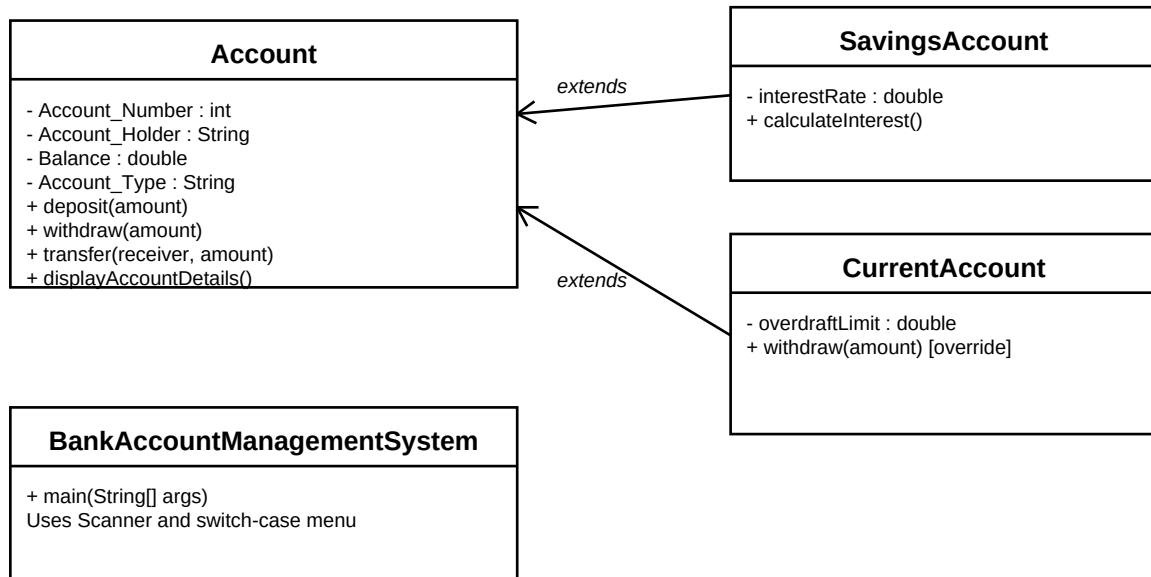
Hardware: Standard PC/Laptop, Core i3/i5 processor or equivalent, 4 GB/8 GB RAM.

Software: Windows/Linux/macOS, JDK 17 or JDK 21, and a text editor or IDE such as VS Code, Eclipse, IntelliJ IDEA, or NetBeans.

3. Algorithm / Logic Description

1. Create the base Account class with account number, holder name, balance, and account type.
2. Initialize account details using a parameterized constructor.
3. Define deposit(), withdraw(), transfer(), and displayAccountDetails() methods.
4. Create SavingsAccount by extending Account and add interestRate and calculateInterest().
5. Create CurrentAccount by extending Account and add overdraftLimit.
6. Override withdraw() in CurrentAccount to permit withdrawal within the overdraft limit.
7. Read Savings Account and Current Account details using Scanner.
8. Display a menu using do-while and switch-case.
9. Perform deposit, withdrawal, transfer, interest calculation, or display operations based on the choice.
10. For transfer, deduct the amount from the sender and add it to the receiver.
11. Repeat until option 7 (Exit) is selected, then close Scanner.

4. UML Class Diagram



5. Source Code

```

import java.util.Scanner;

class Account {
    int Account_Number;
    String Account_Holder;
    double Balance;
    String Account_Type;

    // Parameterized Constructor
    Account(int Account_Number, String Account_Holder,
            double Balance, String Account_Type) {
        this.Account_Number = Account_Number;
        this.Account_Holder = Account_Holder;
        this.Balance = Balance;
        this.Account_Type = Account_Type;
    }

    // Deposit method
    void deposit(double amount) {
        if (amount > 0) {
            Balance += amount;
            System.out.println(amount + " is deposited into your account\n");
            System.out.println("Current account balance is: " + Balance);
        } else {
            System.out.println("Invalid amount! Enter again.");
        }
    }

    // Withdraw method
    void withdraw(double amount) {
        if (amount > 0 && amount <= Balance) {
            Balance -= amount;
            System.out.println(amount + " is withdrawn from your account");
            System.out.println("Current account balance is: " + Balance);
        } else {
            System.out.println("Invalid amount! Enter again.");
        }
    }

    // Transfer method
    void transfer(Account receiver, double amount) {
        if (amount > 0 && amount <= Balance) {
            Balance -= amount;
            receiver.Balance += amount;
            System.out.println("Amount Transferred: " + amount);
        } else {
            System.out.println("Transfer Failed!");
        }
    }

    // Display Account Details
    void displayAccountDetails() {

```

```

        System.out.println("\n---- Account Details ----");
        System.out.println("Account Number : " + Account_Number);
        System.out.println("Holder Name : " + Account_Holder);
        System.out.println("Balance : " + Balance);
        System.out.println("Account Type : " + Account_Type);
    }
}

class SavingsAccount extends Account {
    double interestRate;

    SavingsAccount(int Account_Number, String Account_Holder,
        double Balance, double interestRate) {
        super(Account_Number, Account_Holder, Balance, "Savings");
        this.interestRate = interestRate;
    }

    // Calculate Interest
    void calculateInterest() {
        double interest = Balance * interestRate / 100;
        Balance += interest;
        System.out.println("Interest Added: " + interest);
    }
}

class CurrentAccount extends Account {
    double overdraftLimit;

    CurrentAccount(int Account_Number, String Account_Holder,
        double Balance, double overdraftLimit) {
        super(Account_Number, Account_Holder, Balance, "Current");
        this.overdraftLimit = overdraftLimit;
    }

    // Method Overriding
    @Override
    void withdraw(double amount) {
        if (amount > 0 && amount <= Balance + overdraftLimit) {
            Balance -= amount;
            System.out.println("Amount Withdrawn: " + amount);
        } else {
            System.out.println("Overdraft Limit Exceeded!");
        }
    }
}

public class BankAccountManagementSystem {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.println("===== SAVINGS ACCOUNT =====");
        System.out.print("Enter Account Number: ");
        int sNumber = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Account Holder Name: ");
        String sName = sc.nextLine();

        System.out.print("Enter Initial Balance: ");
        double sBalance = sc.nextDouble();

        System.out.print("Enter Interest Rate: ");
        double rate = sc.nextDouble();

        SavingsAccount savings =
            new SavingsAccount(sNumber, sName, sBalance, rate);

        System.out.println("\n===== CURRENT ACCOUNT =====");
        System.out.print("Enter Account Number: ");
        int cNumber = sc.nextInt();
        sc.nextLine();

        System.out.print("Enter Account Holder Name: ");
        String cName = sc.nextLine();

        System.out.print("Enter Initial Balance: ");
        double cBalance = sc.nextDouble();

        System.out.print("Enter Overdraft Limit: ");
        double overdraft = sc.nextDouble();

        CurrentAccount current =
            new CurrentAccount(cNumber, cName, cBalance, overdraft);

        int choice;

        do {
            System.out.println(
                "\n=====BANK ACCOUNT MENU=====");

```

```

);
System.out.println("1. Deposit");
System.out.println("2. Withdraw");
System.out.println("3. Transfer");
System.out.println("4. Calculate Interest");
System.out.println("5. Display Savings Account");
System.out.println("6. Display Current Account");
System.out.println("7. Exit");
System.out.println("=====");

System.out.print("Enter your choice: ");
choice = sc.nextInt();

switch (choice) {
    case 1:
        System.out.println("\n1. Savings Account");
        System.out.println("2. Current Account");
        System.out.print("Choose Account: ");
        int depositAccount = sc.nextInt();

        System.out.print("Enter Deposit Amount: ");
        double depositAmount = sc.nextDouble();

        switch (depositAccount) {
            case 1:
                savings.deposit(depositAmount);
                break;
            case 2:
                current.deposit(depositAmount);
                break;
            default:
                System.out.println("Invalid Account!");
        }
        break;

    case 2:
        System.out.println("\n1. Savings Account");
        System.out.println("2. Current Account");
        System.out.print("Choose Account: ");
        int withdrawAccount = sc.nextInt();

        System.out.print("Enter Withdrawal Amount: ");
        double withdrawAmount = sc.nextDouble();

        switch (withdrawAccount) {
            case 1:
                savings.withdraw(withdrawAmount);
                break;
            case 2:
                current.withdraw(withdrawAmount);
                break;
            default:
                System.out.println("Invalid Account!");
        }
        break;

    case 3:
        System.out.println("\n1. Savings -> Current");
        System.out.println("2. Current -> Savings");
        System.out.print("Choose Transfer: ");
        int transferChoice = sc.nextInt();

        System.out.print("Enter Transfer Amount: ");
        double transferAmount = sc.nextDouble();

        switch (transferChoice) {
            case 1:
                savings.transfer(current, transferAmount);
                break;
            case 2:
                current.transfer(savings, transferAmount);
                break;
            default:
                System.out.println("Invalid Choice!");
        }
        break;

    case 4:
        savings.calculateInterest();
        break;

    case 5:
        savings.displayAccountDetails();
        break;

    case 6:
        current.displayAccountDetails();
        break;

    case 7:

```

```
        System.out.println(
            "\nThank you for using the Bank System!"
        );
        break;

    default:
        System.out.println("Invalid Choice!");
    }
} while (choice != 7);

sc.close();
}
```

6. Compilation & Execution Commands

Save the file exactly as the public class name:

BankAccountManagementSystem.java

Compile:

javac BankAccountManagementSystem.java

Run:

java BankAccountManagementSystem

7. Output Screen

Representative output using sample input. Actual output depends on the values entered.

```
===== SAVINGS ACCOUNT =====
Enter Account Number: 1001
Enter Account Holder Name: Rahul
Enter Initial Balance: 10000
Enter Interest Rate: 5

===== CURRENT ACCOUNT =====
Enter Account Number: 2001
Enter Account Holder Name: Arjun
Enter Initial Balance: 8000
Enter Overdraft Limit: 3000

=====BANK ACCOUNT MENU=====
1. Deposit
2. Withdraw
3. Transfer
4. Calculate Interest
5. Display Savings Account
6. Display Current Account
7. Exit
=====
Enter your choice: 1

1. Savings Account
2. Current Account
Choose Account: 1
Enter Deposit Amount: 2000
2000.0 is deposited into your account

Current account balance is: 12000.0

Enter your choice: 3
1. Savings -> Current
2. Current -> Savings
Choose Transfer: 1
Enter Transfer Amount: 3000
Amount Transferred: 3000.0

Enter your choice: 4
Interest Added: 450.0

Enter your choice: 5
----- Account Details -----
Account Number : 1001
Holder Name    : Rahul
Balance       : 9450.0
Account Type   : Savings

Enter your choice: 6
----- Account Details -----
Account Number : 2001
Holder Name    : Arjun
Balance       : 11000.0
Account Type   : Current

Enter your choice: 7
Thank you for using the Bank System!
```

8. Result

The Bank Account Management System was successfully implemented. The program demonstrates inheritance, method overriding, constructors, Scanner input, menu-driven switch-case operations, interest calculation, overdraft handling, and transfer of funds between Savings and Current accounts.

Key OOP Concepts Demonstrated: Parameterized Constructor, Inheritance, Method Overriding, Object Creation, Method Reuse, Scanner Input, switch-case, and do-while loop.